

OBJECT ORIENTED PROGRAMMING (25MC409)

I M.C.A II Semester

2025-26 EVEN SEMESTER

**Name of the Faculty : Dr. K. Gayatri
Assistant Professor, Department of CA
VFSTR (Deemed to be University)**

Institute Vision & Mission

VISION:

To evolve in to a centre of excellence in Science & Technology through creative and innovative practices in teaching-learning, towards promoting academic achievement and research excellence to produce internationally accepted, competitive and world class professionals who are psychologically strong & emotionally balanced imbued with social consciousness & ethical values.

MISSION:

To Provide High Quality academic programmes, training activities, research facilities and opportunities supported by continuous industry - institute interaction aimed at promoting employability, entrepreneurship, leadership and research aptitude among students and contribute to the economic and technological development of the region, state and nation.

Department Vision & Mission

VISION:

To emerge as a renowned center in the field of computer applications for providing high quality education and research to produce globally competent professionals with ethical values.

MISSION:

- **M1:** To provide high quality education with cutting-edge curriculum and learner-centered approach.
- **M2:** To establish the culture of innovation and research through industry-academia linkages.
- **M3:** To cultivate ethical and socially-conscious professionals with leadership qualities, who understand the global implications of technology.

PO' s of the Department

- **PO1 (Foundation Knowledge):** Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.
- **PO2 (Problem Analysis):** Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.
- **PO3 (Development of Solutions):** Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- **PO4 (Modern Tool Usage):** Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO' s of the Department

- **PO5 (Individual and Teamwork):** Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- **PO6 (Project Management and Finance):** Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.
- **PO7 (Ethics):** Commit to professional ethics in managing software projects with financial | aspects. Learn to use new technologies for cyber security and insulate customers from malware
- **PO8 (Life-long learning):** Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

PEO' s of the Department

- **PEO1:** Develop cutting-edge technical skills to succeed in career and as an entrepreneur.
- **PEO2 :** Pursue higher education and research with critical thinking and problem solving capabilities
- **PEO3:** Demonstrate ethical, collaborative, and leadership abilities in multi-disciplinary domains.

PSO's of the Department

- PSO1: To provide solutions for complex computational problems with requisite mathematical, programming languages and tools.
- PSO2: To design, develop and analyse software and related services using algorithms, web design, and Big Data Analytics for real time applications.

Module-1

Unit-2

CLASSES AND METHODS: Introduction to classes and methods, objects, Constructors, Overloading Methods and Constructors, Usage of static, Access control, this key word, Garbage Collection.

INHERITANCE: Basics of Inheritance, Types of inheritance, Abstract classes, Interfaces, Usage of final, Usage of super key word, implementing interface, extending interfaces.

Class, Object, Method & Constructor

Syntax: class class-name

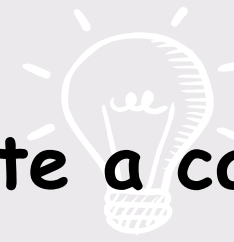
```
{  
    datatype instance-variable1;  
    datatype instance-variable2;  
    -  
    datatype instance-variablen;  
    datatype methodname1(parameter-list)  
    {  
        // method body  
    }  
    -  
    datatype methodnamem(parameter-list)  
    {  
        // method body  
    }  
}
```

Class

A class is a non-primitive data type that contains member variables (data) and member functions (methods) that operates on the variables. A class is declared by the use of the class keyword. The general form of class definition is as follows.



Suppose, we want to create a car



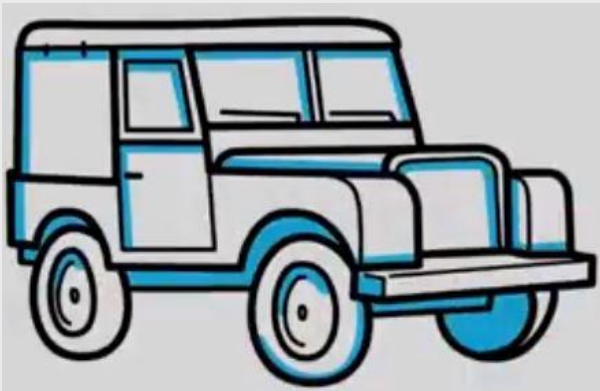
What is the first required thing to create a car?

This plan/blueprint defines the structure of car

We can call this blueprint as a class

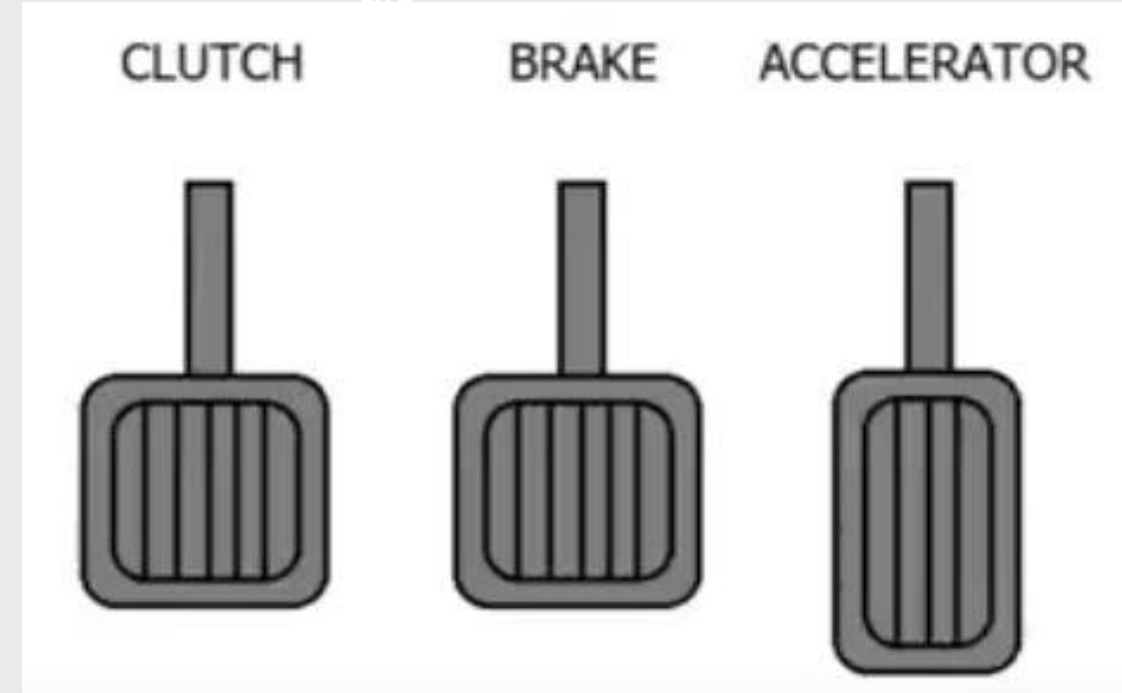


Plan / Blueprint





How we can be able to use these functions/methods?



Car is an Object

Clutch, brake , Accelerator are methods

Blueprint is a class

Methods



Class: Human

Object: Man, Woman, Child

Class: Fruit

Object: Apple, Mango, Banana

Class: Phone

Object: iPhone, Samsung, Nokia

Class: Food

Object: Pizza, Burger, Rice

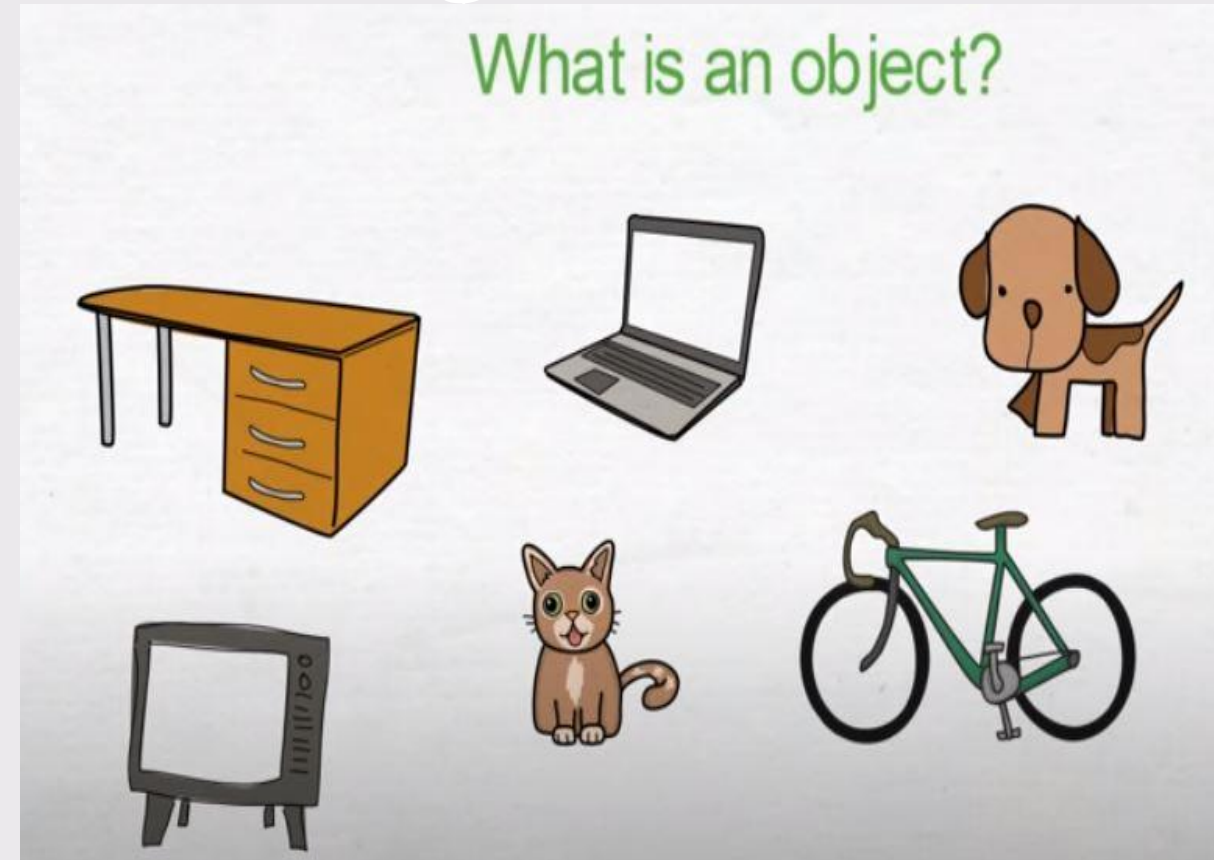
Object

If we look around, we will find lot of objects.....

An object can be related to a real world object.

An object is an instance of a class

Objects have States and behavior



Creating Objects:

- An Object in Java is a block of memory that contains space to store all the instance variables. Creating an object of a class is a two-step process.
 1. Declare a variable of the class type. At this stage, the variable refers to an object only.
 2. Assigning an actual, physical copy of the object and assign it to that variable by means of 'new' operator. The new operator dynamically allocates memory for an object and returns a reference to that object.

Example: `Rectangle rect1;    // reference`
`rect1 = new Rectangle(); // instantiate`
`(or)`
`Rectangle rect1 = new Rectangle();`



Object : dog

Properties

- Name
- Color
- Breed



Variables

Functionality

- Barking
- Wagging tail



Methods



Method

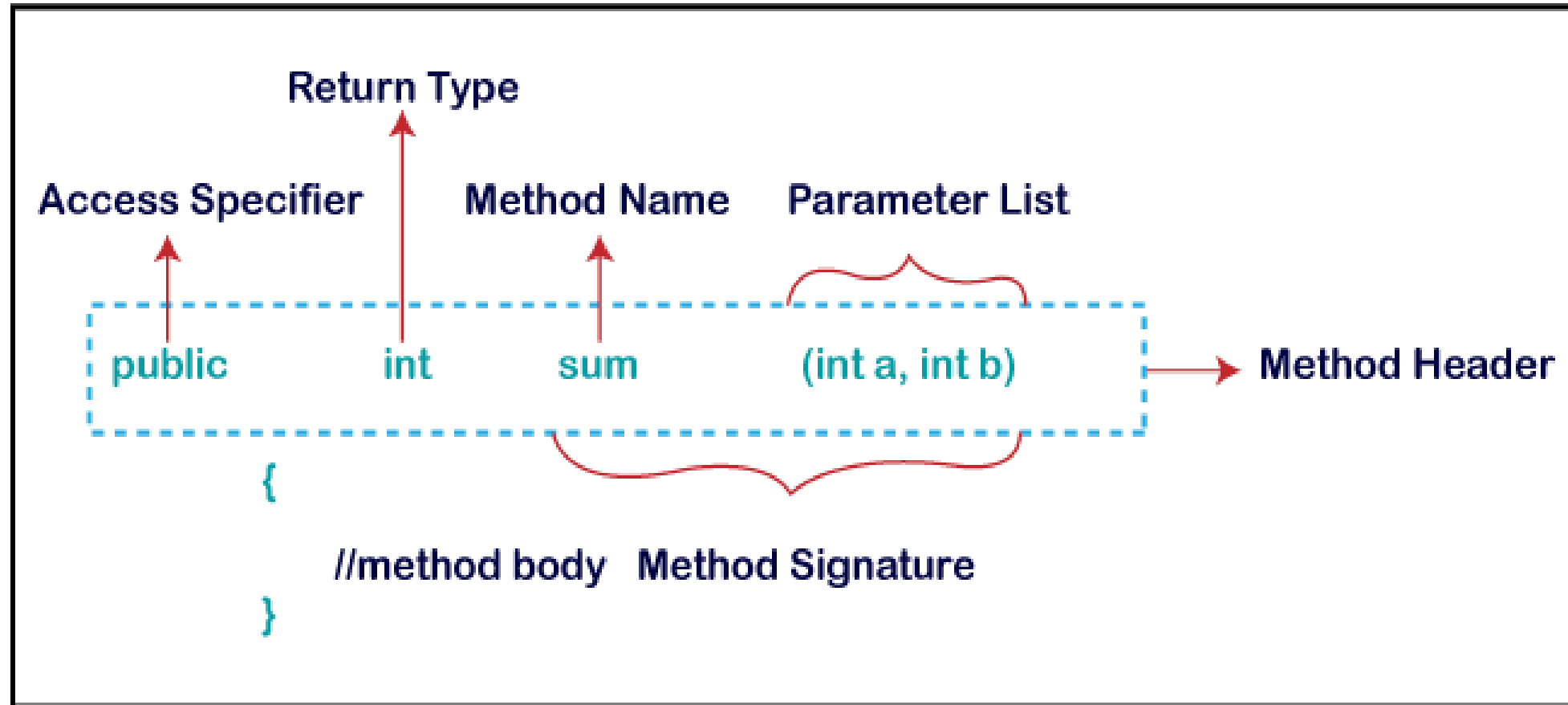


- ✓ In general, a Method is a way to perform some task. A Method is basically a behavior which in turn defines the behavior of a class.
- ✓ A method is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation.
- ✓ We write a method once and use it many times. We do not require to write code again and again.
- ✓ It also provides the easy modification and readability of code, just by adding or removing a chunk of code.
- ✓ The method is executed only when we call or invoke it.

Method Declaration



Method Declaration





```
class sample
{
    public int addNumbers(int a, int b) // create a method
    {
        int sum = a + b;
        return sum; // return value
    }
    public static void main(String[] args)
    {
        int num1 = 25;
        int num2 = 15;
        sample obj = new sample(); // create an object of sample class
        int result = obj.addNumbers(num1, num2); // calling method
        System.out.println("Sum is: " + result);
    }
}
```

Constructor



// create an object of sample class

Constructor



Constructor

A constructor is a special method that is used to initialize an object. The constructor is called when an object of a class is created.

When is a Constructor called?

Each time an object is created using new () keyword at least one constructor (it could be default constructor) is invoked to assign initial values to the data members of the same class.



Note:

- It is called constructor because it constructs the values at the time of object creation.
- It is not necessary to write a constructor for a class.
- It is because java compiler creates a default constructor if your class doesn't have any.
- It doesn't have any return type not even void
- The constructor can be declared as public
- It can be overloaded
- It is normally used to initialize the member of the object
- A constructor cannot be abstract, final, synchronized, and static. We cannot override a constructor. It occupies some space in memory when it is called.

Different types of Constructors:

- ❑ **Default Constructor:** A constructor that accepts no parameters is called the default constructor. If no constructors are defined for a class, the Java system automatically generates the default constructor.
- ❑ **Parameterized Constructor:** A constructor that takes arguments as parameters is called parameterized constructor. If any constructors are defined by a class with the parameters, Java will not create a default constructor for the class.
- ❑ **Copy constructor:** In Java, a copy constructor is a special type of constructor that creates an object using another object of the same Java class. It returns a duplicate copy of an existing object of the class.

Example for default constructor:

```
class Time1 {  
    int hour,min,sec;  
    Time1()  
    {  
        hour=10;  
        min=45;  
        sec=23;  
    }  
    void printdata()  
    {  
        System.out.println(hour+"hours:"+min+"minutes:"+sec+"seconds");  
    }  
}
```

```
class const1{  
    public static void main(String[] args)  
    {  
        Time1 t=new Time1();  
        t.printdata();  
    }  
}
```

Output:

10hours:45minutes:23seconds

Example for Parameterized constructor:

```
class Time1{  
    int hour,min,sec;  
    public Time1(int h,int m,int s)  
    {  
        hour=h;  
        min=m;  
        sec=s;  
    }  
    void printdata()    {  
        System.out.println(hour+"hours:"+min+"minutes:"+sec+"seconds");  
    }  
}
```

```
class const2{  
    public static void main(String[] args) {  
        Time1 t=new Time1(10,9,4);  
        t.printdata();  
    }  
}
```

Output:

10hours:9minutes:4seconds

Example for copy constructor

```
class Rectangle
{
    int length;
    int breadth;
    //constructor to initialize length and breadth of rectangle
    Rectangle(int l, int b)
    {
        length = l;
        breadth = b;
    }
    //copy constructor
    Rectangle(Rectangle obj)
    {
        System.out.println("Copy Constructor Invoked");
        length = obj.length;
        breadth = obj.breadth;
    }
}
```

```
//method to calculate area of rectangle
```

```
int area()
```

```
{
```

```
    return (length * breadth);
```

```
}
```

```
}
```

```
//class to create Rectangle object and calculate area
```

```
class CopyConstructor
```

```
{
```

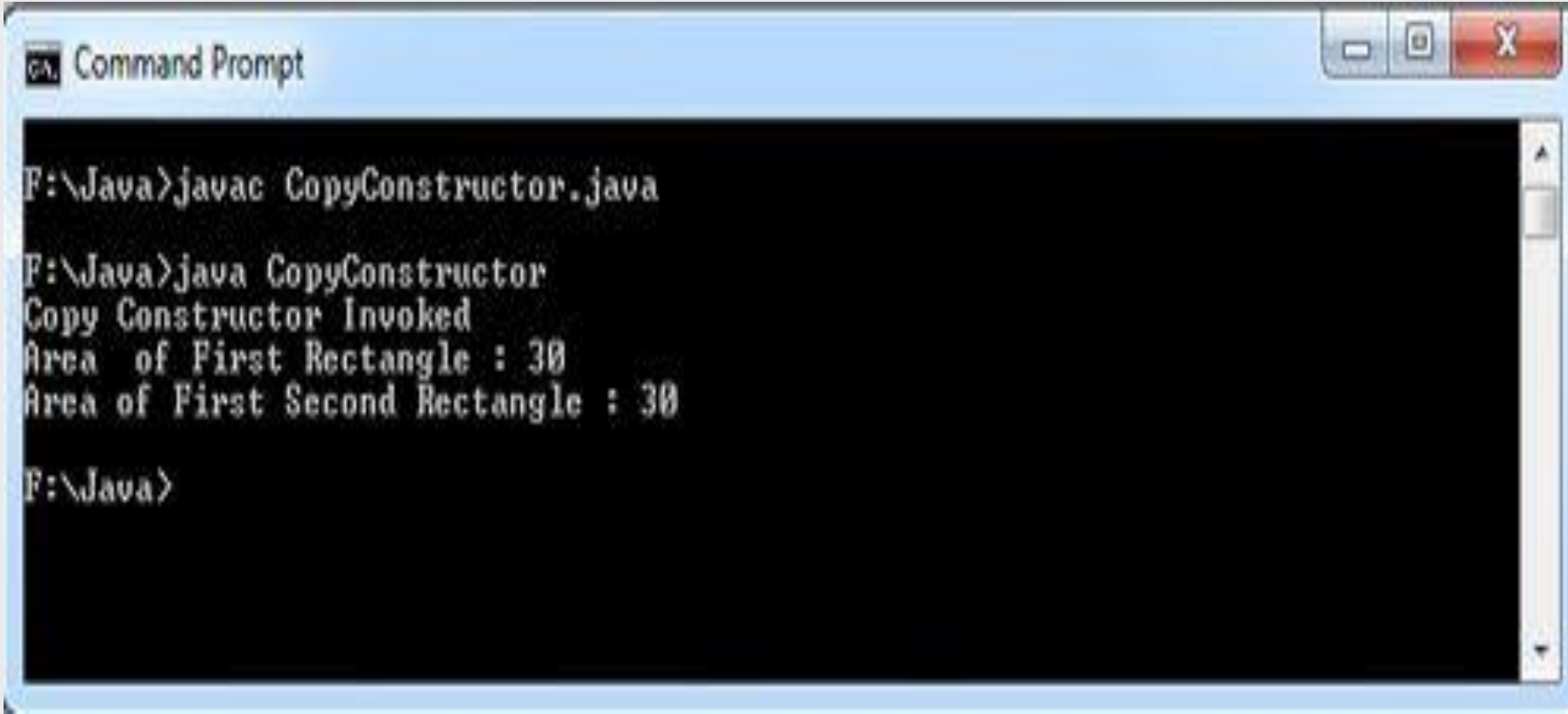
```
    public static void main(String[] args)
```

```
    {
```

```
        Rectangle firstRect = new Rectangle(5,6);
```

```
        Rectangle secondRect= new Rectangle(firstRect);
```

```
System.out.println("Area of First Rectangle : "+ firstRect.area());  
    System.out.println("Area of First Second Rectangle : "+ secondRect.area());  
}  
}
```



```
Command Prompt  
  
F:\Java>javac CopyConstructor.java  
  
F:\Java>java CopyConstructor  
Copy Constructor Invoked  
Area of First Rectangle : 30  
Area of First Second Rectangle : 30  
  
F:\Java>
```

Constructor Overloading

Constructor overloading means having more than one constructor in a class with different parameter lists. It allows objects to be created in different ways.

- Constructor name must be same as class name
- Parameter list must be different
- Constructors do **not** have return types

```
class Student {  
    String name;  
    int registerNo;  
  
    // No-argument constructor  
    Student() {  
        System.out.println("Default constructor called");  
    }  
  
    // Parameterized constructor  
    Student(String name, int registerNo) {  
        this.name = name;  
        this.registerNo = registerNo;  
        System.out.println(name + " and " + registerNo);  
    }  
}
```

```
public static void main(String[] args) {  
    // Calling parameterized constructor  
    Student s1 = new Student("Sudheer", 7004);  
  
    // Calling no-argument constructor  
    Student s2 = new Student();  
}  
}
```

Difference between constructor and method in java

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behaviour of an object.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if you don't have any constructor.	Method is not provided by compiler in any case.
Constructor name must be same as the class name.	Method name may or may not be same as class name.



Static Keyword in Java



Java static variable

If you declare any variable as static, it is known as a static variable

```
class Student
{
    int a =10;
    int b =20;

    public static void main(String[] args)
    {
        System.out.println(a+b);
    }
}
```



```
class Student
{
    static int a =10;
    static int b =20;

    public static void main(String[] args)
    {
        System.out.println(a+b);
    }
}
```



```
class Student
{
    public int add(int a, int b)
    {
        return a+b;
    }

    public static void main(String[] args)
    {
        System.out.println(add(2,3)); //throws error
    }
}
```



```
class Student
{
    public static int add(int a, int b)
    {
        return a+b;
    }

    public static void main(String[] args)
    {

        System.out.println(add(2,3));
    }
}
```



Access Control



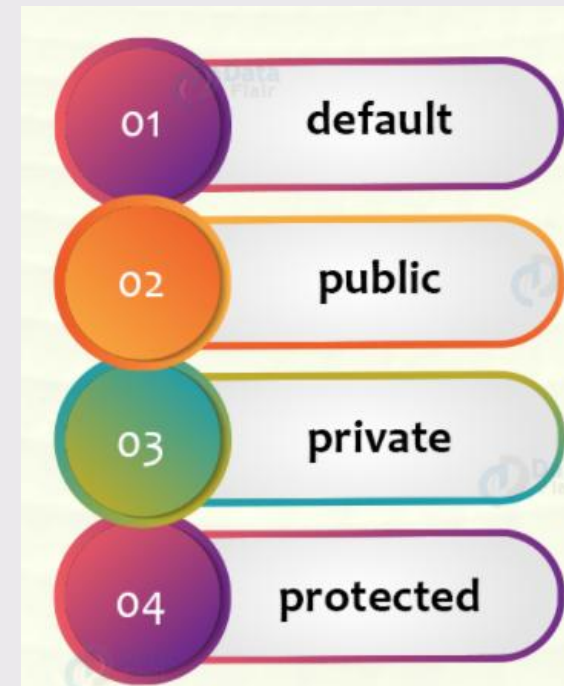
Access Control in Java refers to the mechanism used to restrict or allow access to certain parts of a Java program, such as classes, methods, and variables. Access control determines which classes and objects can access specific codes or data within a program.

Access to members of a class can be controlled using the access modifiers

There are four access modifiers in Java.

They are:

1. public
2. protected
3. default
4. private





Public

- If a class member is “public” then it can be accessed from anywhere.
- The member variable or method is accessed globally. This is the simplest way to provide access to class members.
- However, we should take care of using Public with class variables otherwise anybody can change the values.



Private

- If a class member is “private” then it will be accessible only inside the same class.
- This is the most restricted access and the class member will not be visible to the outer world.
- Usually, we keep class variables as private and methods that are intended to be used only inside the class as private.



Protected

- If class member is “protected” then it will be accessible only to the classes in the *same package* and to the *subclasses*.
- This modifier is less restricted from private but more restricted from public access.
- Usually, we use this keyword to make sure the class variables are accessible only to the subclasses.



Default

- If you don't use any modifier, it is treated as default by default.
- The default modifier is accessible only within package.
- It cannot be accessed from outside the package.



this keyword



The **this** keyword refers to the current object in a method or constructor.

The most common use of the this keyword is to eliminate the confusion between class attributes and parameters with the same name.

(because a class attribute is shadowed by a method or constructor parameter).



Output: 30

```
public class Sample
{
    Sample(int a , int b , int c)
    {
        System.out.println(a+b+c);
    }
    public static void main(String args[])
    {
        Sample s = new Sample(10,10,10);
    }
}
```



Output: 0

```
public class Sample
{
    int x,y,z;

    Sample(int a , int b , int c)
    {
        System.out.println(x+y+z);
    }
    public static void main(String args[])
    {
        Sample s = new Sample(10,10,10);

    }
}
```



```
public class Sample
```

```
{
```

```
    int x,y,z;
```

```
    Sample(int a , int b , int c)
```

```
{
```

```
        x =a;
```

```
        y=b;
```

```
        z=c;
```

```
        System.out.println(x+y+z);
```

```
}
```

```
    public static void main(String args[])
```

```
{
```

```
        Sample s = new Sample(10,10,10);
```

```
}
```

```
}
```

Output: 30



```
public class Sample
```

```
{
```

```
    int x=20,y=20,z=20;
```

```
    Sample(int a , int b , int c)
```

```
{
```

```
        System.out.println(x+y+z);
```

```
}
```

```
    public static void main(String args[])
```

```
{
```

```
        Sample s = new Sample(10,10,10);
```

```
}
```

```
}
```

Output: 60



```
public class Sample
{
    int a,b,c;

    Sample(int a , int b , int c)
    {
        a=a;
        b=b;
        c=c;
        System.out.println(a+b+c);
    }
    public static void main(String args[])
    {
        Sample s = new Sample(10,10,10);
    }
}
```

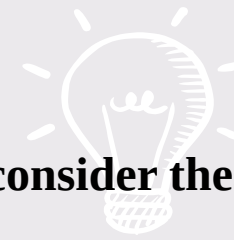
Assignment to itself



Output: 30

```
public class Sample
{
    int a,b,c;

    Sample(int a , int b , int c)
    {
        this.a=a;
        this.b=b;
        this.c=c;
        System.out.println(a+b+c);
    }
    public static void main(String args[])
    {
        Sample s = new Sample(10,10,10);
    }
}
```



```
public class Sample  
{
```

```
    int a=20,b=20,c=20;
```

```
    Sample(int a , int b , int c)
```

```
{
```

```
        System.out.println(a+b+c);
```

```
}
```

```
    public static void main(String args[])
```

```
{
```

```
        Sample s = new Sample(10,10,10);
```

```
}
```

```
}
```

How to consider these values?

```
System.out.println(this.a+this.b+this.c);
```

```
a= this.a;
```

```
b=this.b;
```

```
c=this.c;
```

Garbage Collection

Garbage Collection

Garbage Collection in Java is an automatic memory management process that **reclaims unused memory** by destroying objects that are no longer referenced by a program. It helps prevent memory leaks and improves application reliability.

Java Heap Generations

- **Young Generation**- New objects are created and allocated in this memory area.
- **Old Generation**- Long-lived objects that survive multiple garbage collection cycles are stored here.

Types of Garbage Collection Activities

Minor (Incremental) Garbage Collection

- Minor Garbage Collection occurs when unreachable objects in the Young Generation are removed.
- Since most objects are short-lived, this type of GC happens frequently and is fast.

Major (Full) Garbage Collection

- Major or Full Garbage Collection removes unreachable objects from the Old Generation.
- It occurs less frequently but takes more time because it processes long-lived objects.

How Garbage Collection Works

- It identifies which objects are still in use (referenced) and which are not in use (unreferenced).
 - It removes the objects that are unreachable (no longer referenced).
 - The programmer does not need to mark objects to be deleted explicitly.
- Garbage collection is implemented within the JVM.

Example on Garbage collection

```
import java.lang.Object;
class Garbage {
    int x,y;
    void setdata(int a,int b) {
        x=a;
        y=b;
    }
    void printdata() {
        System.out.println("x="+x+" y="+y);
    }
    protected void finalize() {
        System.out.println("finalizer");
    }
}
```

```
class Mg {  
    public static void main(String[] args) {  
        Garbage obj=new Garbage();  
        Garbage obj1=new Garbage();  
        obj.setdata(10,20);  
        obj.printdata();  
        obj1=null;  
        obj=null;  
        System.gc(); // method to call garbage collector  
        obj1.setdata(2,3); //error  
        obj.printdata();  
        obj1.printdata();  
    }  
}
```



Inheritance

- Inheritance can be defined as the process where one class acquires the properties (methods and fields) of another class.
- The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).
- To inherit from a class, use the “*extends*” keyword



Child Class

The class that extends the features of another class is known as child class, sub class or derived class.

Parent Class

The class whose properties and functionalities are used (inherited) by another class is known as parent class, super class or Base class.



Three (3) important things

- Inheritance represents the IS-A relationship which is also known as a *parent-child* relationship.
- Code reusability (Biggest Advantage)
- extends key word (For implementing inheritance)



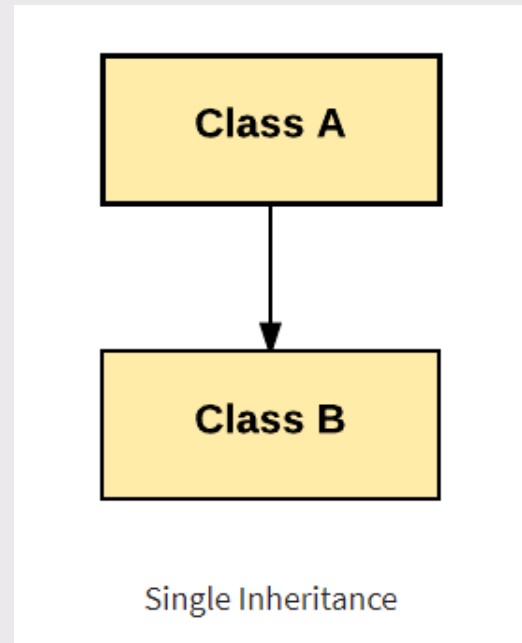
Types of Inheritance

1. Single Inheritance
2. Multi-level Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance



Single Inheritance

- In single inheritance, one class inherits the properties of another class.
- It enables a derived class to inherit the properties and behavior from a single parent class.
- In Single Inheritance one class extends another class (one class only).





```
class A
{
    public void methodA()
    {
        System.out.println("Method A in class A");
    }
}

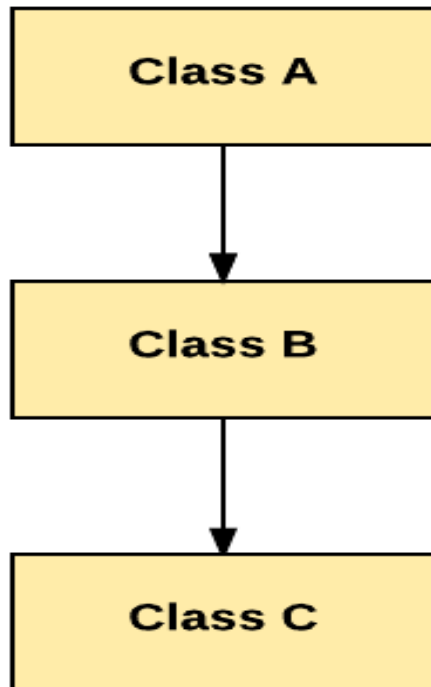
public class Sample extends A
{
    public static void main(String args[])
    {
        Sample s = new Sample();
        s.methodA ();
    }
}
```



Multi-level Inheritance



- When a class is derived from a class which is also derived from another class.
- A class having more than one parent class but at different levels, such type of inheritance is called Multilevel Inheritance.



Multilevel Inheritance

- Class B inherits the properties of class A
- Class C inherits the properties of class B.
- Here A is the parent class for B
- Class B is the parent class for C.
- So in this case class C implicitly inherits the properties and methods of class A along with Class B.

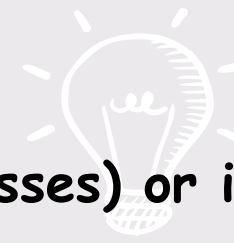


```
class A
{
    public void methodA()
    {
        System.out.println("Method A in class A");
    }
}
class B extends A
{
    public void methodB()
    {
        System.out.println("Method B in class B");
    }
}
```

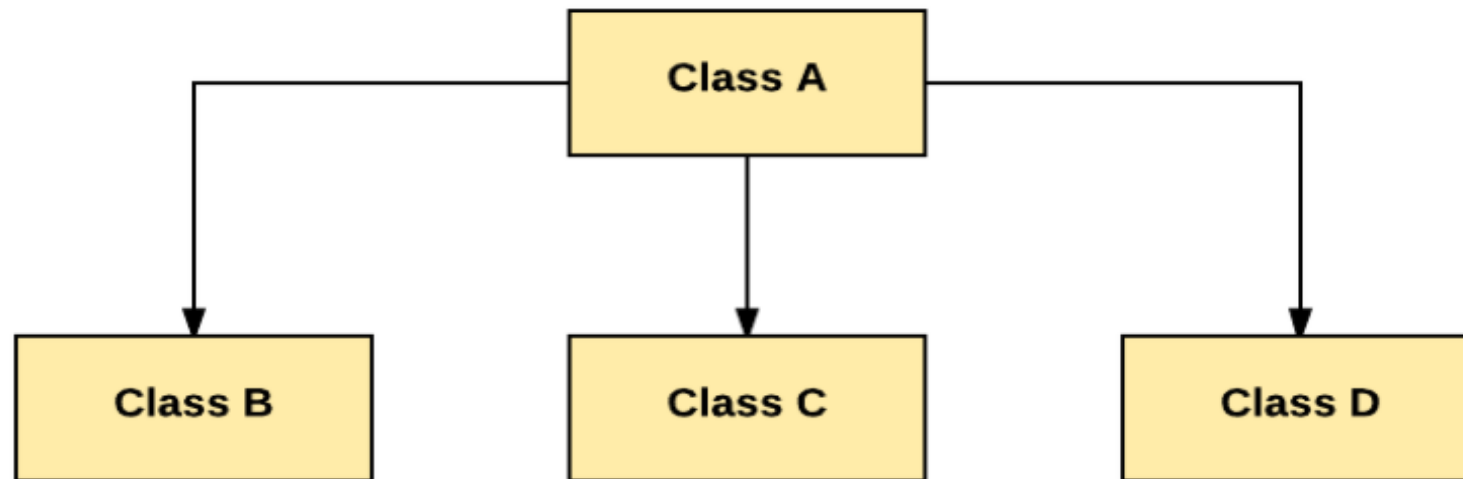
```
public class Sample extends B
{
    public static void main(String args[])
    {
        Sample s = new Sample();
        s.methodA ();
        s.methodB ();
    }
}
```



Hierarchical Inheritance



- When a class has more than one child classes (subclasses) or in other words,
- More than one child classes have the same parent class, then such kind of inheritance is known as hierarchical.



Hierarchical Inheritance

As per above example, Class B, C, and D inherit the same class A.



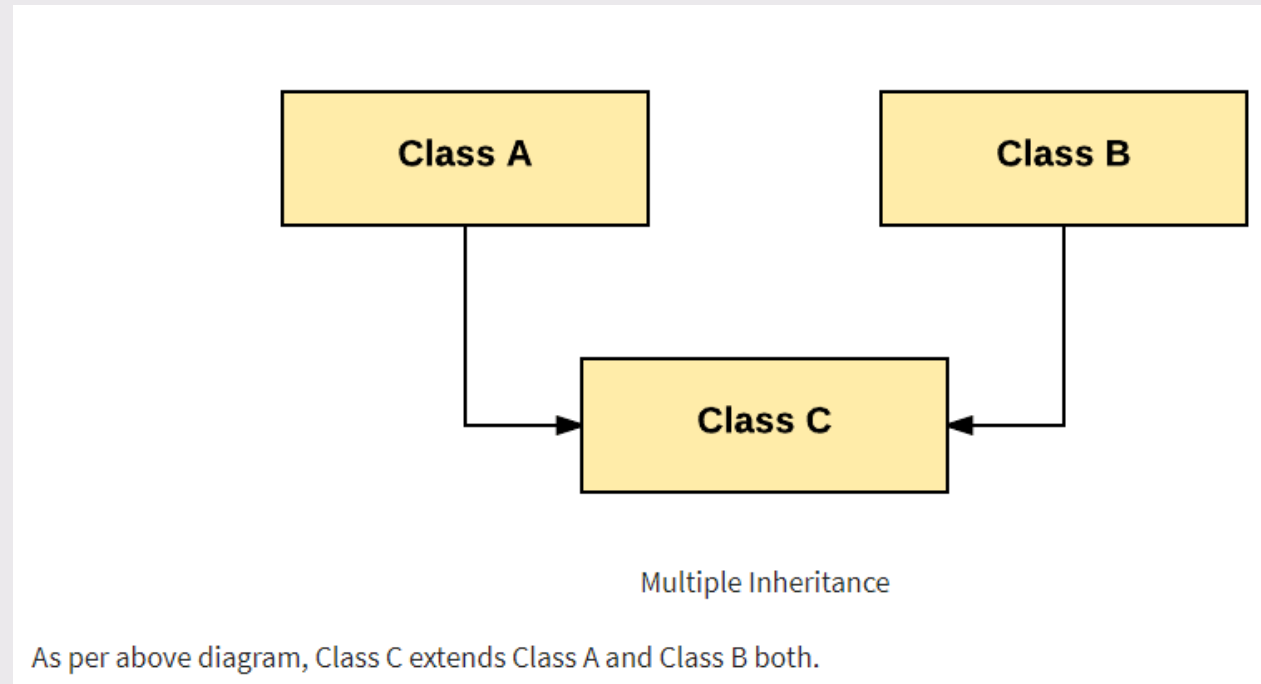
```
class A
{
    public void methodA()
    {
        System.out.println("Method A in class A");
    }
}
class B extends A
{
    public void methodB()
    {
        System.out.println("Method B in class B");
    }
}
```

```
public class Sample extends A  
{  
    public static void main(String args[])  
    {  
        Sample s = new Sample();  
        B b = new B();  
        s.methodA ();  
        b.methodB ();  
    }  
}
```



Multiple Inheritance

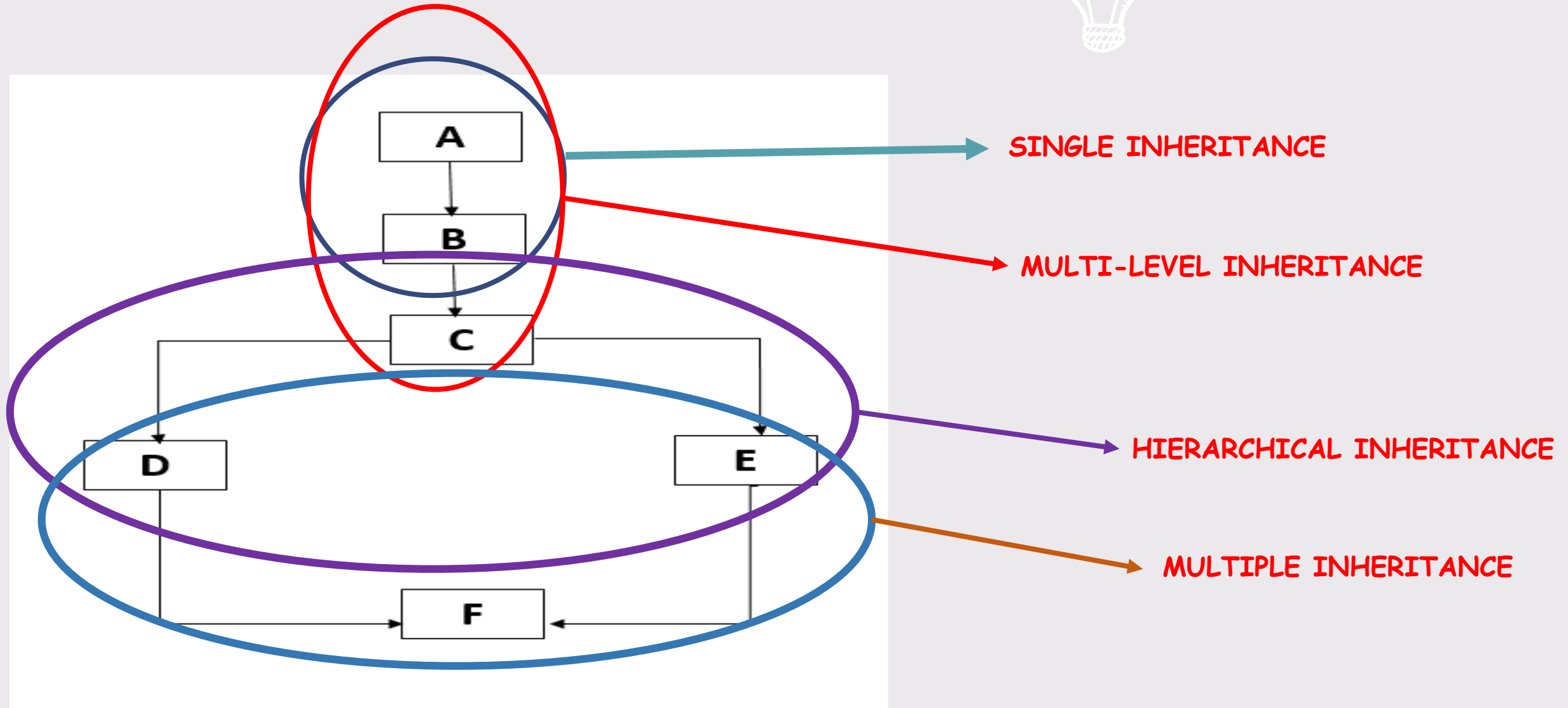
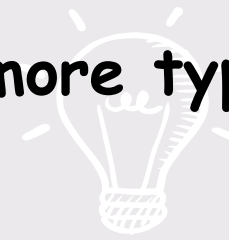
- Multiple inheritance refers to the process where one child class tries to extend more than one parent class.
- In Multiple Inheritance, one class extending from more than one class.





Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance





Method Overloading



- ✓ **Method overloading** is a concept that allows to declare multiple methods with same name but different parameters in the same class

Three ways to overload a method

Number of parameters

Data type of parameters

Sequence of Data type of parameters



1. Number of parameters

add (int, int)
add (int, int, int)

2. Data type of parameters

add (int, int)
add (int, float)

3. Sequence of Data type of parameters

add (int, float)
add (float, int)

```
public class Sample
{
    public void add(int a, int b, int c)
    {
        System.out.println(a+b+c);
    }
    public void add(int a, int b)
    {
        System.out.println(a+b);
    }
    public void add(int a, double b)
    {
        System.out.println(a+b);
    }
    public static void main(String args[])
    {
        Sample s = new Sample();
        s.add(10,20,30);
        s.add(10,20);
        s.add(10,1.2);
    }
}
```





Constructor Overloading



Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

```
public class Sample
```

```
{
```

```
    Sample(String name , String email, String mobile)
```

```
    {
```

```
        System.out.println(name + "," + email + "," + mobile);
```

```
    }
```

```
    Sample(String name , String email)
```

```
    {
```

```
        System.out.println(name + "," + email);
```

```
    }
```

```
    Sample(String name)
```

```
    {
```

```
        System.out.println(name);
```

```
    }
```

```
    public static void main(String args[])
```

```
    {
```

```
        Sample s = new Sample("vignan", "v@gmail.com", "9878767656");
```

```
        Sample s1 = new Sample("vignan", "v@gmail.com");
```

```
        Sample s2 = new Sample("vignan");
```

```
    }
```





Method Overriding



- Declaring a method in sub class which is already present in parent class is known as method overriding.
- Overriding is done so that a child class can give its own implementation to a method which is already provided by the parent class.
- In this case the method in parent class is called overridden method and the method in child class is called overriding method.
- Method overriding is used to provide the specific implementation of a method which is already provided by its Parent class.



Rules for Java Method Overriding

- The method must have the **same name** as in the parent class.
- The method must have the **same parameter** as in the parent class.
- There must be an IS-A relationship (**inheritance**).

```

class A
{
    public void add()
    {
        int a=10,b=10;
        System.out.println (a+b);
    }
}

class B extends A
{
    public void add()
    {
        int a=10,b=10,c=10;
        System.out.println (a+b+c);
    }
}

```



```

public class Sample
{
    public static void main(String args[])
    {
        A a = new A();
        B b = new B();
        a.add();
        b.add();
    }
}

```

```

A a = new B();
a.add();

```

Parent reference can be used to hold child class reference (JVM will take care at execution time)



Abstraction & Abstract Class



- Abstraction is a process of hiding the implementation details and showing only functionality to the user.
- In Java, abstraction is accomplished using Abstract class, Abstract methods, and Interfaces.

Abstract Class



- It is a type of class in Java that declare one or more abstract methods. These classes can have abstract methods as well as non-abstract methods.
- An abstract class is a class that contains at least one abstract method.
- An abstract class is a class that cannot be instantiated (we cannot create objects of an abstract class and we can't call the methods directly with object).
- In Java, we use the abstract keyword to declare an abstract class.
- Abstract class = not proper implementation

ABSTRACT METHOD



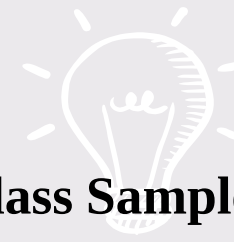
- It is a method that has just the method definition but does not contain implementation.
- A method without a body is known as an Abstract Method.
- It must be declared in an abstract class.
- We use the same keyword abstract to create abstract methods.



```
abstract class vehicle
{
    public void add()
    {
        System.out.println("add method");
    }
    public abstract void message();
}
```

```
public class Sample extends vehicle
{
    public void message()
    {

    }
    public static void main(String args[])
    {
        Sample s = new Sample();
        vehicle v = new vehicle();
        s.add();
    }
}
```



```
abstract class Animal
```

```
{  
    public abstract void sound();  
}
```

```
class Dog extends Animal
```

```
{  
    public void sound()  
    {  
        System.out.println("bow-bow");  
    }  
}
```

```
class Cat extends Animal
```

```
{  
    public void sound()  
    {  
        System.out.println("meow-meow");  
    }  
}
```

```
public class Sample
```

```
{  
    public static void main(String args[])  
    {  
        Dog g = new Dog();  
        Cat c = new Cat();  
        g.sound();  
        c.sound();  
    }  
}
```



Interface



- Interface looks like a class but it is not a class.
- Interfaces are declared by specifying a keyword "interface"
- An interface can have methods and variables just like the class but the methods declared in interface are by default abstract (only method signature but no body).
- The interface in Java is a *mechanism to achieve abstraction*. There can be only abstract methods in the Java interface, not method body.



What is the use of interface in Java?

- Since methods in interfaces do not have body, they have to be implemented by the class before you can access them. The class that implements interface must implement all the methods of that interface.
- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.



```
interface MyInterface
```

```
{  
    public void method1();  
    public void method2();  
}
```

```
public class Sample implements MyInterface
```

```
{  
    public void method1()  
    {  
        System.out.println("implementation of method1");  
    }  
    public void method2()  
    {  
        System.out.println("implementation of method2");  
    }  
}
```

```
public static void main(String args[])
```

```
{  
    Sample s = new Sample();  
    s.method1();  
    s.method2();  
}  
}
```



Extending Interfaces



An interface can extend another interface in the same way that a class can extend another class.

The “extends” keyword is used to extend an interface, and the child interface inherits the methods of the parent interface.



```
interface MyInterface1
```

```
{  
    public void method1();  
}
```

```
interface MyInterface2 extends MyInterface1
```

```
{  
    public void method2();  
}
```

```
public class Sample implements MyInterface2
```

```
{  
    public void method1()  
    {  
        System.out.println("implementation of method1");  
    }  
    public void method2()  
    {  
        System.out.println("implementation of method2");  
    }  
    public static void main(String args[])  
    {  
        Sample s = new Sample();  
        s.method1();  
        s.method2();  
    }  
}
```

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4) Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.

Abstract class	Interface
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword <code>?extends?</code> .	An interface class can be implemented using keyword <code>?implements?</code> .
8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.
9) Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Drawable{ void draw(); }</pre>



final keyword



- Final keyword is a modifier applicable for classes, methods and variables.
- If a class declared as final, we can't extend that class(we cannot create child class for that class)
- If a method declared as final, we can't override that method in child class.
- If a variable declared as final, then it will become constant and we can't perform re-assignment for that variable.

A **final** is a keyword used for **three** purposes. They are

a) final as constant: A variable can be declared as constant with the keyword final. It must be initialized while declaring. Once we initialized the variable with final keyword it cannot be modified in the program. This is similar to the const in C/C++.

Example: final int x = 10;

x = 45 //error

b) final to prevent overriding:

A method that is declared as final cannot be overridden in a subclass method. If the methods that are declared as private are implicitly final.

Example: class A

```
{
    final void show( )
    {
        System.out.println("Hello");
    }
}
class B extends A
{
    void show( )
    {    //error
        System.out.println("Hai");
    }
}
```

c) final to prevent inheritance:

The class that is declared as final implicitly declares all of its methods as final. The class cannot be extended to its subclass.

Example: final class A

```
{  
    void show( )  
    {  
        System.out.println("Hello");  
    }  
}  
class B extends A                //error  
{  
    void show( )  
    {  
        System.out.println("Hai");  
    }  
}
```



Super Keyword



Super Keyword

- The keyword super can be used to access any data member of the parent class.
- Super keyword can be used at variable, method and constructor level.
- The super keyword in Java is a reference variable which is used to refer immediate parent class object.



- The `super` keyword refers to superclass (parent) objects.
- It is used to call superclass methods, and to access the superclass constructor.
- The most common use of the `super` keyword is to eliminate the confusion between superclasses and subclasses that have methods with the same name.

class A

```
{  
    public void add()  
    {  
        System.out.println("Add method from Class A");  
    }  
}
```

public class Sample extends A

```
{  
    public void add()  
    {  
        System.out.println("Add method from Class Sample");  
    }  
}
```

public void display()

```
{  
    add();  
}
```

public static void main(String args[])

```
{  
    Sample s = new Sample();  
    s.display();  
}
```

```
}
```

super.add();



Usage of java super Keyword

- ❑ super can be used to refer immediate parent class instance variable.
- ❑ super can be used to invoke immediate parent class method.
- ❑ super() can be used to invoke immediate parent class constructor.

1) super is used to refer immediate parent class instance variable.

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

```
class Animal
{
    String color="white";
}
class Dog extends Animal
{
    String color="black";
    void printColor()
    {
        System.out.println(color);
        //prints color of Dog class
    }
}
```

```
System.out.println(super.color);  
    //prints color of Animal class  
    }  
}  
class Super1{  
    public static void main(String args[])  
    {  
        Dog d=new Dog();  
        d.printColor();  
    }  
}
```

Output:

Black

white

2) super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
class Animal
{
    void eat(){
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void eat(){
        System.out.println("eating bread...");
    }
    void bark(){
        System.out.println("barking...");
    }
}
```

```
void work()
{
    super.eat();
    bark();
}

class TestSuper2
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.work();
    }
}
```

Output:

eating...barking...

3) super is used to invoke parent class constructor

The super keyword can also be used to
invoke the parent class constructor.

Let's see a simple example:

```
class Animal
{
    Animal()
    {
        System.out.println("animal is created");
    }
}
class Dog extends Animal
{
    Dog()
    {
        super();
    }
}
```

```
        System.out.println("dog is created");
    }
}
class TestSuper3
{
    public static void main(String args[])
    {
        Dog d=new Dog();
    }
}
```

Output:

animal is created

dog is created